

<통신시스템 캡스톤 디자인1>

4차보고서

-4.14(4.16 변경)-

주요계획

1. USB 웹캠(Logitech C920) 연결 ✓
2. 카메라 인식 확인 ✓
3. OpenCV 기반 영상 캡처 및 이미지 저장✓
4. 라즈베리파이에서 캡처한 이미지를 PC서버로 전송✓
5. HTTP POST 방식 업로드 기능 구현✓
6. 라이브 스트리밍 구현✓

목표: 영상 입력 환경 구축 완료
이미지 전송 기능 구현

-팀원-

오석준

정민영

황다원



1. 체크리스트 및 역할분담

DAY	주요 계획	목표	비고
3.31	Raspberry Pi OS 기반 환경 점검 SSH 원격 접속 구성 네트워크 및 IP 확인 Python/pip/venv 환경 구성, YOLO 설치 및 실행 가능성 검토	라즈베리파이 개발 환경 구축 완료 YOLO 실행 오류 (Illegal instruction) 확인	완료
4.7	PC 기반 FastAPI 서버 구축 기본 API(GET/POST) 작성 라즈베리파이↔서버 통신 확인	라즈베리파이와 PC 서버 간 통신 가능 상태 확보	완료
4.14	USB 웹캠(Logitech C920) 연결 카메라 인식 확인 OpenCV 기반 영상 캡처 및 이미지 저장	영상 입력 환경 구축 완료	완료
	라즈베리파이에서 캡처한 이미지를 PC 서버로 전송 HTTP POST 방식 업로드 기능 구현 라이브 스트리밍 기능 구현	이미지 전송 기능 구현	
5.5	PC 서버에서 YOLOv8n/v8s 실행 환경 구성 이미지 기반 객체 탐지 테스트	객체 탐지 기능 1차 구현	
5.12	탐지 결과 기반 침입 판단 로직 구현 ROI(제한 영역) 설정	단순 인식에서 침입 판단 기능으로 확장	
5.19	등록 얼굴 데이터 수집, 얼굴 검출 및 특정 추출 방식 설계, 등록 인물과 미등 록 인물을 구분하기 위한 얼굴 인식 구 조 설계, 결과 저장을 위한 DB 구조 설 계	얼굴 인식 엔진 및 데 이터 저장 체계 확립	
5.26	Opencv 기반 얼굴 검출과 특징 비교 기 능 구현, 등록 인물/미등록 인물 판별 테 스트, yolo 사람 탐지 결과와 얼굴 인식 결과를 결합한 판별하는 로직을 구현	등록 인물과 외부인 구 분 기능 구현, 객체 탐 지와 얼굴 인식을 결합 한 판별 구조 확인	
6.2	이벤트 정의 및 알림 기능 구현 침입 발생 시 로그/알림 생성	이벤트 기반 시스템 완성	
6.9	전체 시스템 통합 테스트 오류 수정 및 성능 개선	1학기 1차 완성본 도출	

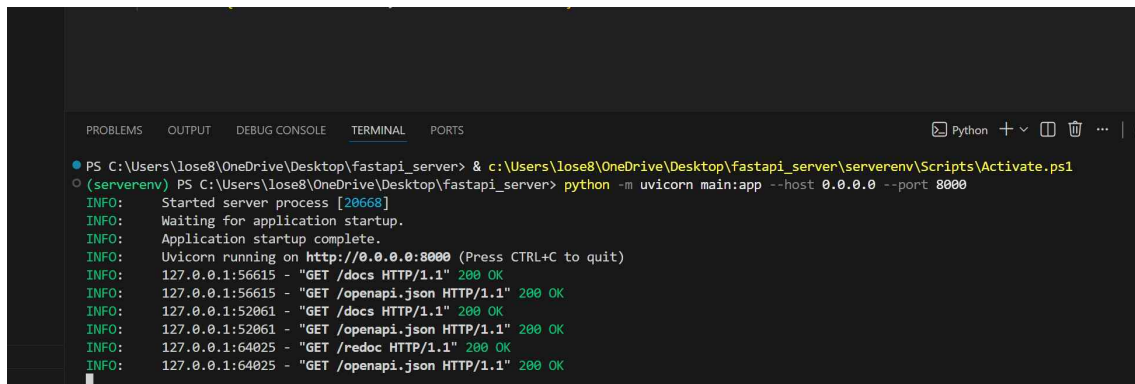
회차	오석준 처리, YOLO, 서버	o/x	정민영 입력, 전송, 통신	0/x	황다원 DB, UI, 데이터	o/x
2차 3.31	1. Raspberry Pi 환경 점검 및 YOLO 실행 가능성 검토 2. Python 개발 환경 구축 및 YOLO 설치 후 실행 테스트 수행 - 세부업무: 기존 OS 상태 확인, 가상 환경 구성, YOLO 설치 및 import 테스트, 실행 오류 (Illegal instruction) 발생 원인 분석	0	1. Raspberry Pi 네트워크 및 IP 확인을 통한 통신 환경 점검 2. YOLO 실행 결과 검토 및 보고서 작성, 발표 준비 - 세부업무: SSH 접속 및 IP 확인, 네트워크 상태 점검, 오류 로그 분석, 구조 변경 방향 문서화	0	1. Raspberry Pi 관련 자료 조사 및 OS 설치, SSH 환경 구성 2. 개발 환경 구축 지원 및 프로젝트 준비 사항 정리 - 세부업무: Raspberry Pi OS 설치, SSH 설정, 개발 환경 구성 지원, 물품 조사 및 정리	0
3차 4.7	1. FastAPI 서버 구축 및 API 구현 2. 서버 요청 처리 로직 구성 - 세부업무: 서버 실행 환경 구성, GET/POST API 작성, 요청 처리 흐름 설계, 서버 실행 테스트	0	1. Raspberry Pi → 서버 데이터 전송 구현 2. 통신 테스트 및 연결 검증 - 세부업무: HTTP 요청 코드 작성, 서버 API 호출, 응답 확인, 통신 오류 점검	0	1. 데이터 흐름 구조 설계 2. API 데이터 형식 정의 - 세부업무: 요청/응답 JSON 구조 설계, 전체 시스템 흐름 문서화, 통신 구조 정리	0
4차 4.14	1. 웹캠과 라즈베리 파이를 연결하여 카메라 입력 환경을 구축 2. mjpg-streamer를 이용한 웹 스트리밍 환경을 구성	0	1. Raspberry Pi 이미지 캡처 및 서버 전송 구조 구현 2. 전송 이미지 해상도 및 용량 조정을 통한 입력 데이터 최적화 - 세부업무: OpenCV 기반 캡처, 이미지 전송 코드 구현, 해상도 조정, 전송 속도 테스트	0	1. 이벤트 데이터 구조 설계 2. DB 반영 및 저장 기능 구현 세부 업무: event_type 정의, 결과 저장 구현	0
5차 5.5	1. YOLO 결과 처리 로직 구현 2. 탐지 데이터 변환 - 세부업무:		1. 이미지 전송 주기 설정 2. 전송 안정성 테스트		1. ROI 기반 판단 기능 개발 2. 데이터 저장 기능 구현	

	bounding box 좌표 추출, detected 값 생성, 데이터 구조 변환		- 세부업무: 전송 간격 조정, 반복 전송 테스트, 서버 응답 확인		- 세부업무: 테이블 설계, insert 로직 구현, 저장 데이터 검증, DB 수정	
6차 5.12	1. 이벤트 생성 로직 구현 2. 중복 방지 처리 - 세부업무: 상태 변화 감지, 이벤트 발생 조건 설정, 시간 기반 제어		1. 연속 전송 테스트 2. 시스템 동작 검증 - 세부업무: 반복 전송 테스트, 이벤트 발생 흐름 확인		1. 이벤트 DB 구축 2. 데이터 관리 기능 구현 - 세부업무: events 테이블 생성, 저장 로직 구현, 데이터 확인	
7차 5.19	1. YOLO 모델(v8n, v8s) 성능 비교 및 최적 모델 선정 2. 객체 탐지 정확도 및 처리 속도 분석 - 세부업무: 모델 비교, 처리 시간 측정, 정확도 분석		1. 이미지 전송 속도 및 시스템 FPS 측정 2. 전송 효율 및 네트워크 성능 분석 - 세부업무: FPS 계산, latency 측정, 전송 주기 비교, 전송 구조 최적화 및 네트워크 병목 분석		1. DB 조회 성능 개선 및 인덱스 적용 2. 데이터 증가에 따른 DB 처리 속도 분석 - 세부업무: 인덱스 설정, 조회 속도 비교, 성능 분석	
8차 5.26	1. API 기능 확장 2. 데이터 처리 기능 구현 - 세부업무: 로그 조회 API, 이벤트 조회 API, JSON 응답 처리		1. 데이터 수집 및 테스트 2. 통신 안정성 검증 - 세부업무: 다양한 환경 테스트, 전송 오류 확인		1. UI 구현 2. 데이터 시각화 - 세부업무: 로그 화면 구성, 이미지 출력, UI 구성	
9차 6.2	1. 서버 구조 개선 2. 동시 요청 처리 - 세부업무: 멀티 요청 처리, 서버 안정성 확보		1. 다중 Pi 구성 2. 동시 전송 테스트 - 세부업무: Pi 2대 설정, 동시 요청 테스트		1. 데이터 구분 구조 설계 2. 카메라별 관리 기능 구현 - 세부업무: camera_id 추가, 데이터 분리 저장	

2. 전체 진행 순서

- 🕒 PC 에서 FastAPI 서버 실행 (1)
- 🕒 PC IPv4 주소 확인 (2)
- 🕒 Raspberry Pi 에서 웹캠 연결 및 장치 인식 확인 (3)
- 🕒 기본 촬영 도구로 이미지 캡처 확인 (4)
- 🕒 OpenCV,requests 설치 (5)
- 🕒 OpenCV 기반 이미지 캡처 및 전송 코드 작성 (6)
- 🕒 PC 서버 기록과 저장 폴더를 확인하여 이미지 전송 성공 여부 검증 (7)
- 🕒 정적 이미지 전송 이후 실시간 영상 확인을 위한 MJPG 스트리밍 검증 (8)
- 🕒 FastAPI와 MySQL 연결(9)

1. PC 에서 FastAPI 서버 실행 (1 of 9)



```
PS C:\Users\lose8\OneDrive\Desktop\fastapi_server> & c:\Users\lose8\OneDrive\Desktop\fastapi_server\serverenv\Scripts\Activate.ps1
(serverenv) PS C:\Users\lose8\OneDrive\Desktop\fastapi_server> python -m uvicorn main:app --host 0.0.0.0 --port 8000
INFO: Started server process [20668]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO: 127.0.0.1:56615 - "GET /docs HTTP/1.1" 200 OK
INFO: 127.0.0.1:56615 - "GET /openapi.json HTTP/1.1" 200 OK
INFO: 127.0.0.1:52061 - "GET /docs HTTP/1.1" 200 OK
INFO: 127.0.0.1:52061 - "GET /openapi.json HTTP/1.1" 200 OK
INFO: 127.0.0.1:64025 - "GET /redoc HTTP/1.1" 200 OK
INFO: 127.0.0.1:64025 - "GET /openapi.json HTTP/1.1" 200 OK
```

pc에서 fastapi 서버를 실행한다.

raspberrypi가 pc서버로 이미지를 업로드해야 하므로 서버가 외부 요청을 받을 수 있도록 0.0.0.0 호스트로 열어두어야 함.

부연설명(저번 3차 보고서 내용)

1. `python -m uvicorn main:app --host 0.0.0.0 --port 8000`

◇ `python -m uvicorn` = uvicorn 서버를 실행

◇ `main: app` = main.py 파일 안의 app이라는 객체를 실행(`app = FASTAPI( )`)

◇ `--host 0.0.0.0` : 같은 네트워크의 다른 장치에서도 접속 가능하게

연결

◇--port 8000 : 8000 번 포트에서 요청을 받겠다.

= 기존 코드 uvicorn main:app만 써도

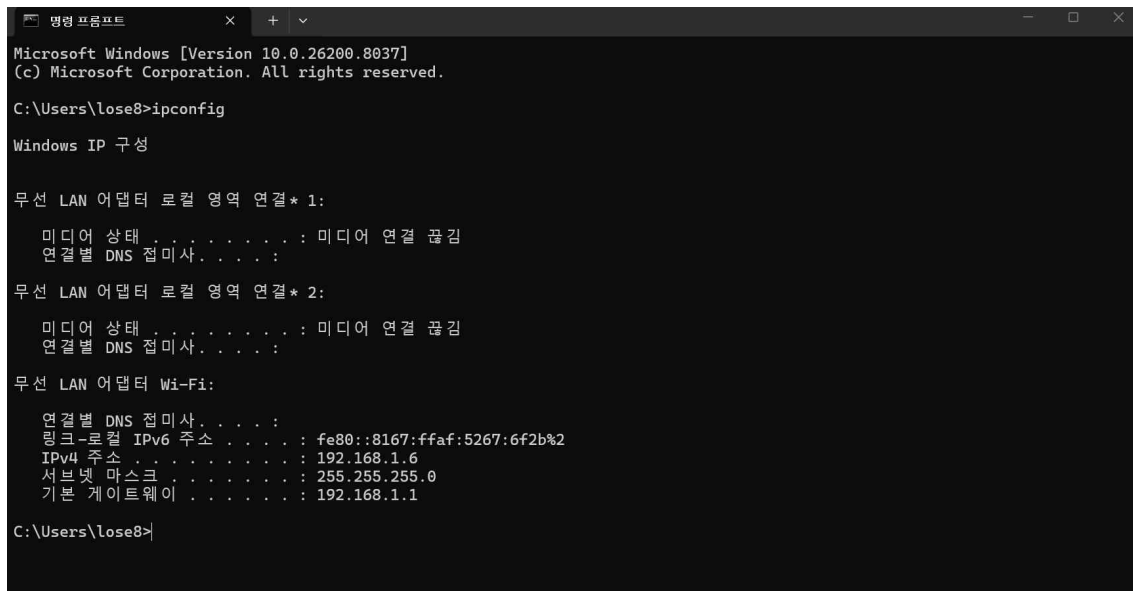
pc(http://127.0.0.1:8000/docs)에서만 접속 가능하지만,

--host 0.0.0.0 -port 8000를 포함하여 같은 네트워크의 라즈베리
파이(http://192.168.0.113:8000/docs)에서도 접속 가능하게 할 수
있다.

2. PC IPv4 주소 확인 (2 of 9)

ipconfig를 이용하여 pc의 실제 ip주소를 사용해야 함.

(pc ip 주소: 192.168.1.6)



```
Microsoft Windows [Version 10.0.26200.8037]
(c) Microsoft Corporation. All rights reserved.

C:\Users\lose8>ipconfig

Windows IP 구성

무선 LAN 어댑터 로컬 영역 연결* 1:

    미디어 상태 . . . . . : 미디어 연결 끊김
    연결별 DNS 접미사 . . . . :

무선 LAN 어댑터 로컬 영역 연결* 2:

    미디어 상태 . . . . . : 미디어 연결 끊김
    연결별 DNS 접미사 . . . . :

무선 LAN 어댑터 Wi-Fi:

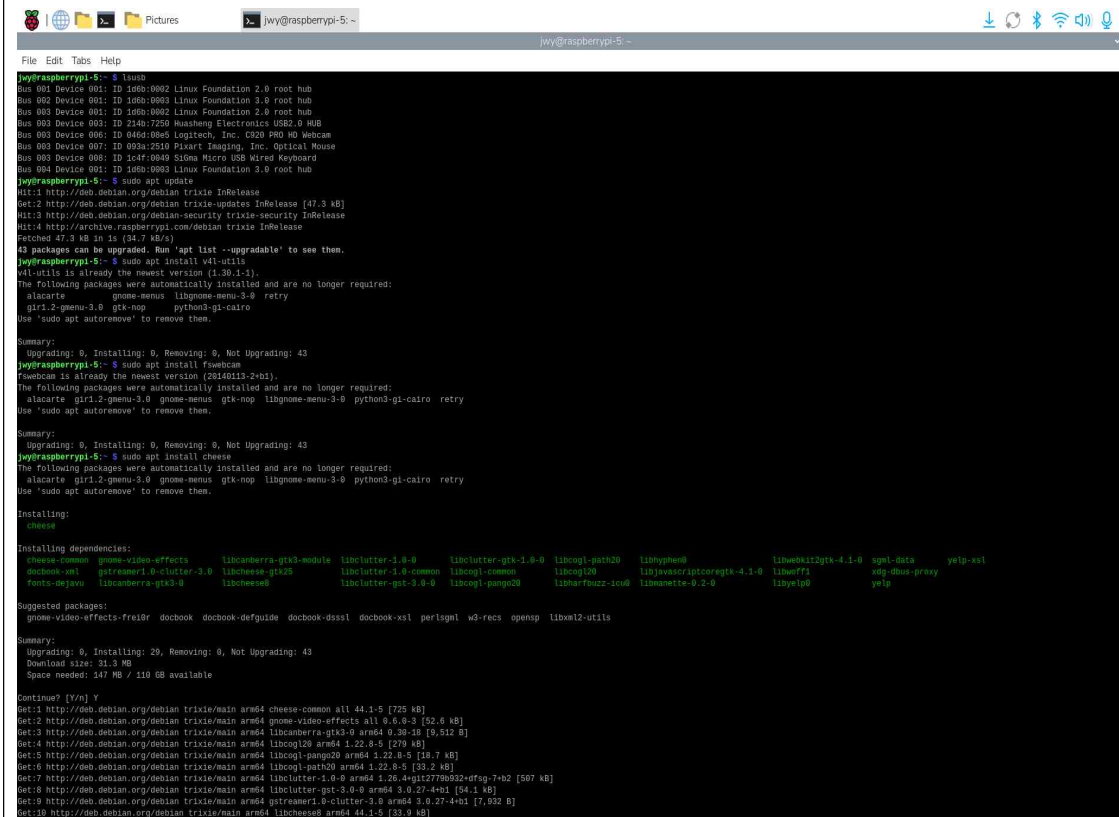
    연결별 DNS 접미사 . . . . :
    링크-로컬 IPv6 주소 . . . . : fe80::8167:ffaf:5267:6f2b%2
    IPv4 주소 . . . . . : 192.168.1.6
    서브넷 마스크 . . . . . : 255.255.255.0
    기본 게이트웨이 . . . . . : 192.168.1.1

C:\Users\lose8>
```

3. Raspberry Pi 에서 웹캠 연결 및 장치 인식 확인 (3 of 9)

Raspberry Pi 5에 USB 웹캠(Logitech C920)을 연결한 뒤, 리눅스가 이를 영상 입력 장치로 정상 인식하는지 먼저 확인한다.

라즈베리파이 cmd 명령어 입력 과정 전체 사진



```
juvy@raspberrypi-5: ~ $ lsusb
Bus 001 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
Bus 002 Device 001: ID 1d6b:0003 Linux Foundation 3.0 root hub
Bus 003 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
Bus 003 Device 003: ID 240:7200 Washung Electronics USB2.0 xHUB
Bus 003 Device 006: ID 046d:08e5 Logitech, Inc. C920 PRO HD Webcam
Bus 003 Device 007: ID 093a:2510 Pixart Imaging, Inc. Optical Mouse
Bus 003 Device 008: ID 1d4f:0040 Sigma Micro USB Wired Keyboard
Bus 004 Device 001: ID 1d6b:0003 Linux Foundation 3.0 root hub

juvy@raspberrypi-5:~$ sudo apt update
Hit:1 http://deb.debian.org/debian trixie InRelease
Hit:2 http://deb.debian.org/debian trixie-updates InRelease [47.3 kB]
Hit:3 http://deb.debian.org/debian-security trixie-security InRelease
Hit:4 http://archive.raspberrypi.com/debian trixie InRelease
Fetched 27.2 kB in 1s (31.7 kB/s)
43 packages can be upgraded. Run 'apt list --upgradable' to see them.

juvy@raspberrypi-5:~$ sudo apt install v4l-utils
v4l-utils is already the newest version (1.30.1-1).
The following packages were automatically installed and are no longer required:
  alacarte          gnome-menus        libgnome-menu-3-0  retry
  gir1.2-gmenu-3.0  gtk-nop            python3-gi-cairo
Use 'sudo apt autoremove' to remove them.

Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 43
juvy@raspberrypi-5:~$ sudo apt install libcamera
libcamera is already the newest version (20140113-2+b1).
The following packages were automatically installed and are no longer required:
  alacarte gir1.2-gmenu-3.0 gnome-menus  gtk-nop  libgnome-menu-3-0  python3-gi-cairo  retry
Use 'sudo apt autoremove' to remove them.

Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 43
juvy@raspberrypi-5:~$ sudo apt install cheese
The following packages were automatically installed and are no longer required:
  alacarte gir1.2-gmenu-3.0 gnome-menus  gtk-nop  libgnome-menu-3-0  python3-gi-cairo  retry
Use 'sudo apt autoremove' to remove them.

Installing:
  cheese

Installing dependencies:
  cheese-common  gnome-video-effects  libcambrera-gtk3-module  libclutter-3.0-0  libclutter-gtk-3.0-0  libcogl-pango20  libhyphen0  libkhi2gtk-4-1-0  sgml-data  yelp-xsl
  docbook-xsl  gststreamer1.0-clutter-3.0  libcheese-gtk25  libclutter-1.0-common  libcogl-common  libcogl100  libjavascriptcoregtk-4.1-0  libheif1  org.debian.proxy  yelp
  fonts-dejavu  libcambrera-gtk3-0  libcheese6  libclutter-gst-3.0-0  libcogl-pango20  libharfbuzz-icu0  libmanette-0.2-0  libyelp0  yelp

Suggested packages:
  gnome-video-effects-frei0r  docbook  docbook-defguide  docbook-dsssl  docbook-xsl  perlsgml  w3-recs  opensp  libxml2-utils

Summary:
  Upgrading: 0, Installing: 29, Removing: 0, Not Upgrading: 43
  Download size: 31.3 MB
  Space needed: 147 MB / 110 GB available

Continue? [Y/n] Y
Get:1 http://deb.debian.org/debian trixie/main arm64 cheese-common all 44.1-5 [725 kB]
Get:2 http://deb.debian.org/debian trixie/main arm64 gnome-video-effects all 0.6.8-3 [52.6 kB]
Get:3 http://deb.debian.org/debian trixie/main arm64 libcambrera-gtk3-0 arm64 0.38-10 [9,512 B]
Get:4 http://deb.debian.org/debian trixie/main arm64 libcogl128 arm64 1.22.8-5 [279 kB]
Get:5 http://deb.debian.org/debian trixie/main arm64 libcogl-pango20 arm64 1.22.8-5 [18.7 kB]
Get:6 http://deb.debian.org/debian trixie/main arm64 libcogl-pango20 arm64 1.22.8-5 [33.2 kB]
Get:7 http://deb.debian.org/debian trixie/main arm64 libclutter-1.0-0 arm64 1.26.4-g12778b022-dfsg-7+b2 [507 kB]
Get:8 http://deb.debian.org/debian trixie/main arm64 libclutter-gst-3.0-0 arm64 3.0.0-27-4+b1 [64.1 kB]
Get:9 http://deb.debian.org/debian trixie/main arm64 gststreamer1.0-clutter-3.0 arm64 3.0.27-4+b1 [7,832 B]
Get:10 http://deb.debian.org/debian trixie/main arm64 libcheese6 arm64 44.1-5 [133.0 kB]
```

lsusb

```
jwy@raspberrypi-5:~$ lsusb
Bus 001 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
Bus 002 Device 001: ID 1d6b:0003 Linux Foundation 3.0 root hub
Bus 003 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
Bus 003 Device 003: ID 214b:7258 Huasheng Electronics USB2.0 HUB
Bus 003 Device 006: ID 046d:08e5 Logitech, Inc. C920 PRO HD Webcam
Bus 003 Device 007: ID 093a:2510 Pixart Imaging, Inc. Optical Mouse
Bus 003 Device 008: ID 1c4f:0049 Sigma Micro USB Wired Keyboard
Bus 004 Device 001: ID 1d6b:0003 Linux Foundation 3.0 root hub
```

라즈베리파이에 연결된 USB 장치들의 목록과 상세 정보를 확인.

(카메라가 제대로 인식되었는지 확인하기 위해 다음 명령어를 실행합니다.)

Bus 003 device 006: ID 046d:08e5 Logitech, inc. C920 PRO HD webcam -> 웹캠이 제대로 연결됨.

이외에 다른 usb장치들인 usb허브나 마우스, 키보드 등이 연결되어 있는 것을 알 수 있다.

sudo apt update

sudo apt install v4l-utils

```
jwy@raspberrypi-5:~$ sudo apt update
Hit:1 http://deb.debian.org/debian trixie InRelease
Get:2 http://deb.debian.org/debian trixie-updates InRelease [47.3 kB]
Hit:3 http://deb.debian.org/debian-security trixie-security InRelease
Hit:4 http://archive.raspberrypi.com/debian trixie InRelease
Fetched 47.3 kB in 1s (34.7 kB/s)
43 packages can be upgraded. Run 'apt list --upgradable' to see them.
jwy@raspberrypi-5:~$ sudo apt install v4l-utils
v4l-utils is already the newest version (1.30.1-1).
The following packages were automatically installed and are no longer required:
  alacarte          gnome-menus  libgnome-menu-3-0  retry
  gir1.2-gmenu-3.0  gtk-nop     python3-gi-cairo
Use 'sudo apt autoremove' to remove them.

Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 43
```

라즈베리파이는 기본적으로 Video4Linux(V4L2)를 지원하고 있습니다.

패키지가 누락 될수도 있다고 생각하여 설치하였지만 카메라 관련 도구인 v4l-utils는 이미 설치되어 있고 가장 최신 버전으로 새로 설치할 필요가 없었습니다.

sudo apt install fswebcam

```
jwy@raspberrypi-5:~$ sudo apt install fswebcam
fswebcam is already the newest version (20140113-2+b1).
The following packages were automatically installed and are no longer required:
  alacarte gir1.2-gmenu-3.0 gnome-menus gtk-nop libgnome-menu-3-0 python3-gi-cairo retry
Use 'sudo apt autoremove' to remove them.

Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 43
```

명령어 기반 캡처 도구의 설치 명령어(기본 캡처)

이미 설치되어 있음.

sudo apt install cheese

```
jwy@raspberrypi-5:~$ sudo apt install cheese
The following packages were automatically installed and are no longer required:
  alacarte gir1.2-gmenu-3.0 gnome-menus gtk-nop libgnome-menu-3-0 python3-gi-cairo retry
Use 'sudo apt autoremove' to remove them.

Installing:
  cheese

Installing dependencies:
  cheese-common  gnome-video-effects  libcamberra-gtk3-module  libclutter-1.0-0  libclutter-gtk-1.0-0  lib
  docbook-xsl  gstreamer1.0-clutter-3.0  libcheese-gtk25  libclutter-1.0-common  libcrogl-common  lib
  fonts-dejavu  libcamberra-gtk3-0  libcheese0  libclutter-gst-3.0-0  libcrogl-pango20  lib

Suggested packages:
  gnome-video-effects-frei0r docbook docbook-defguide docbook-dsssl docbook-xsl perlsgml w3-recs opensp libx

Summary:
  Upgrading: 0, Installing: 29, Removing: 0, Not Upgrading: 43
  Download size: 31.3 MB
  Space needed: 147 MB / 119 GB available

Continue? [Y/n] Y
```

GUI 기반 테스트 도구의 설치 명령어(라즈베리파이 라이브 스트리밍)

설치 O

ls /dev/video*

```
jwy@raspberrypi-5:~$ ls /dev/video*
/dev/video0 /dev/video19 /dev/video21 /dev/video23 /dev/video25 /dev/video27 /dev/video29 /dev/video31 /dev/video33 /dev/video35
/dev/video1 /dev/video20 /dev/video22 /dev/video24 /dev/video26 /dev/video28 /dev/video30 /dev/video32 /dev/video34
```

현재 시스템에서 영상 장치로 인식된 것을 전부 보여달라는 명령어
리눅스에서는 카메라나 영상 관련 장치가 /dev/video 형태로 잡힘.
요즘 웹캠에는 비디오 입력과 디코딩 관련 장치 등이 있어서 같이 잡

힐수 있다.

v4l2-ctl -list-devices

```
jwy@raspberrypi-5:~$ v4l2-ctl --list-devices
pispbe (platform:1800880000.pisp_be):
/dev/video20
/dev/video21
/dev/video22
/dev/video23
/dev/video24
/dev/video25
/dev/video26
/dev/video27
/dev/video28
/dev/video29
/dev/video30
/dev/video31
/dev/video32
/dev/video33
/dev/video34
/dev/video35
/dev/media8
/dev/media1

rpi-hevc-dec (platform:rpi-hevc-dec):
/dev/video19
/dev/media2

HD Pro Webcam C920 (usb-xhci-hcd.1-1.4):
/dev/video0
/dev/video1
/dev/media3
```

USB 카메라가 제대로 인식되었는지 확인하기 위한 명령어

HD PRO webcam C920

video 0이랑 1을 주로 쓰는데 역할이 있고 0을 촬영용으로 쓸거다.
참조)

pispbe는 라즈베리파이 내부 이미지 처리 파이프라인 관련 장치(영상 처리 백엔드-웹캠 촬영에 직접 쓰는 장치가 아니다.)

rpi -hevc-dec는 비디오 디코딩 관련 하드웨어 장치

HD pro webcam C920(video 0,1,3)

웹캠 촬영에 쓰임

0은 실제 촬영용 영상 입력 장치로 많이 쓰임

1은 보조 스트림이나 다른 기능(제어)용으로 많이 쓰임(보조역할)

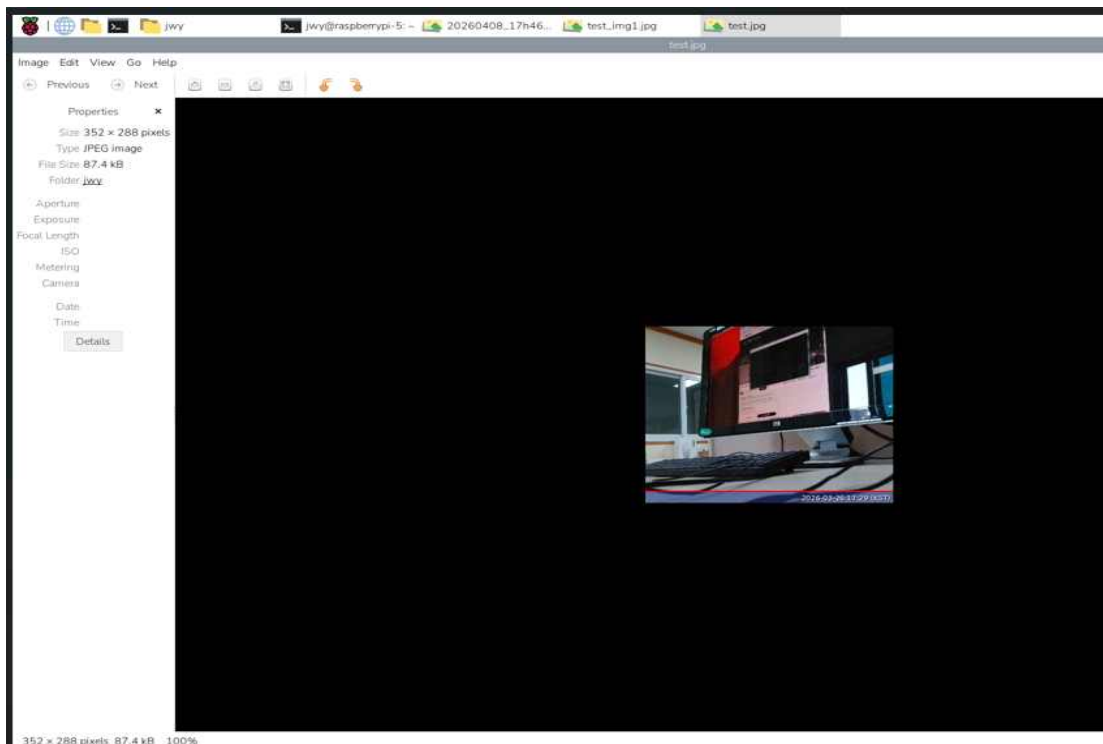
3은 카메라 내부 연결 구조를 보여주는 미디어 컨트롤러(카메라 장치 구성 정보)

4. 기본 촬영 도구로 이미지 캡처 확인 (4 of 9)

fswebcam test_img.jpg

```
jwy@raspberrypi-5:~ $ fswebcam test_img.jpg
--- Opening /dev/video0...
Trying source module v4l2...
/dev/video0 opened.
No input was specified, using the first.
Adjusting resolution from 384x288 to 352x288.
--- Capturing frame...
Captured frame in 0.00 seconds.
--- Processing captured image...
Writing JPEG image to 'test_img.jpg'.
```

이미지 캡처



command창을 보면

adjusting resolution from 384*288 to 352*288

원래 해상도도 낮은데, 저장이 size가 352*288 pixel로 됐다.(기본값)

->해상도를 높여야 함.

v4l2-ctl -device=/dev/video0 -list-formats-ext

```
jwy@raspberrypi-5:~$ v4l2-ctl --device=/dev/video0 --list-formats-ext
ioctl: VIDIOC_ENUM_FMT
Type: Video Capture

[0]: 'YUYV' (YUYV 4:2:2)
    Size: Discrete 640x480
        Interval: Discrete 0.033s (30.000 fps)
        Interval: Discrete 0.042s (24.000 fps)
        Interval: Discrete 0.050s (20.000 fps)
        Interval: Discrete 0.067s (15.000 fps)
        Interval: Discrete 0.100s (10.000 fps)
        Interval: Discrete 0.133s (7.500 fps)
        Interval: Discrete 0.200s (5.000 fps)
    Size: Discrete 160x90
        Interval: Discrete 0.033s (30.000 fps)
        Interval: Discrete 0.042s (24.000 fps)
        Interval: Discrete 0.050s (20.000 fps)
        Interval: Discrete 0.067s (15.000 fps)
        Interval: Discrete 0.100s (10.000 fps)
        Interval: Discrete 0.133s (7.500 fps)
        Interval: Discrete 0.200s (5.000 fps)
    Size: Discrete 1280x720
        Interval: Discrete 0.100s (10.000 fps)
        Interval: Discrete 0.133s (7.500 fps)
        Interval: Discrete 0.200s (5.000 fps)
    Size: Discrete 1600x896
        Interval: Discrete 0.133s (7.500 fps)
        Interval: Discrete 0.200s (5.000 fps)
    Size: Discrete 1920x1080
        Interval: Discrete 0.200s (5.000 fps)
    Size: Discrete 2560x1472
        Interval: Discrete 0.500s (2.000 fps)
[1]: 'MJPG' (Motion-JPEG, compressed)
    Size: Discrete 640x480
        Interval: Discrete 0.033s (30.000 fps)
        Interval: Discrete 0.042s (24.000 fps)
        Interval: Discrete 0.050s (20.000 fps)
        Interval: Discrete 0.067s (15.000 fps)
        Interval: Discrete 0.100s (10.000 fps)
        Interval: Discrete 0.133s (7.500 fps)
        Interval: Discrete 0.200s (5.000 fps)
    Size: Discrete 1920x1080
        Interval: Discrete 0.033s (30.000 fps)
        Interval: Discrete 0.042s (24.000 fps)
        Interval: Discrete 0.050s (20.000 fps)
        Interval: Discrete 0.067s (15.000 fps)
        Interval: Discrete 0.100s (10.000 fps)
        Interval: Discrete 0.133s (7.500 fps)
        Interval: Discrete 0.200s (5.000 fps)
```

웹캠이 지원하는 해상도 확인

1.YUYV:비압축(저압축)에 가까운 원시 영상 포맷(압축 없음)

장점: 해상도가 2560x1472 까지 지원 가능, 원본 데이터라서 손실 없음.

단점: 데이터 양이 크고, 고해상도일수록 대역폭 부담 크다.

2.MJPG:motion jpeg 압축 포맷 (프레임을 JPEG로 압축하여 USB 대역폭을 적게 사용)

장점:압축으로 데이터가 가벼움. 1080p 실사용으로 현실적, 스트리밍에도 유리함.(1920x1080까지 지원 가능)

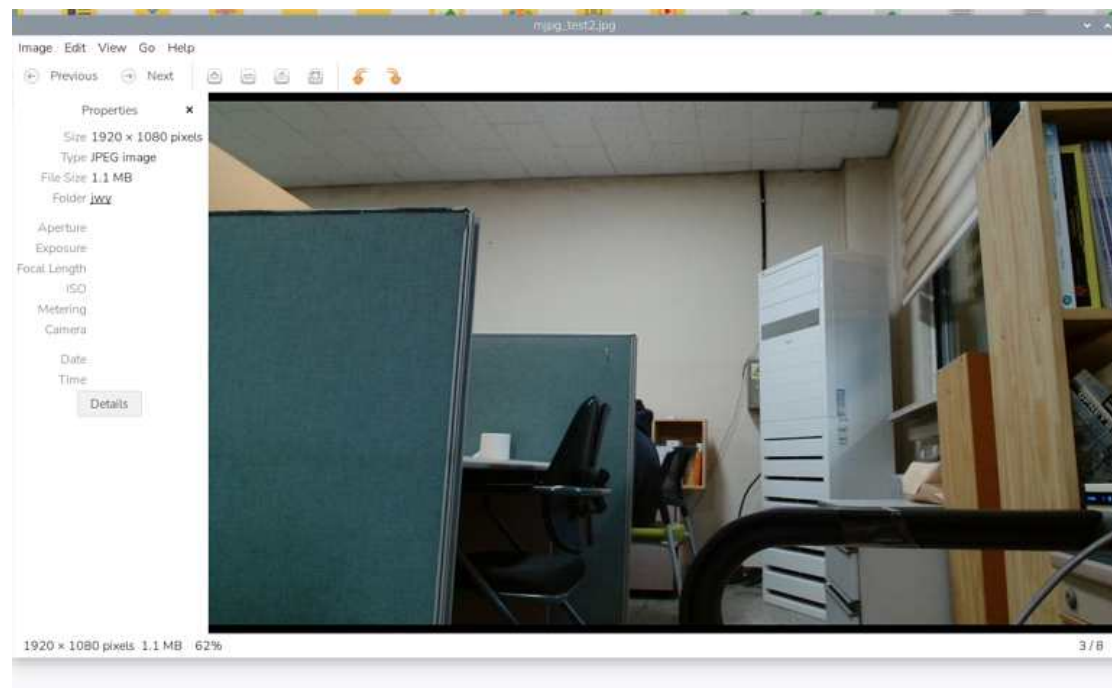
단점:압축이 들어가서 손실 가능성 있음.

->프로젝트에서는 대역폭 부담이 적고 데이터가 가벼우며, CCTV와 같은 실시간 용도로 적합한 것은 MJPG

```
fswebcam -d /dev/video0 -p MJPEG -r 1920x1080 --skip 20 --no-banner mjpg_test2.jpg
```

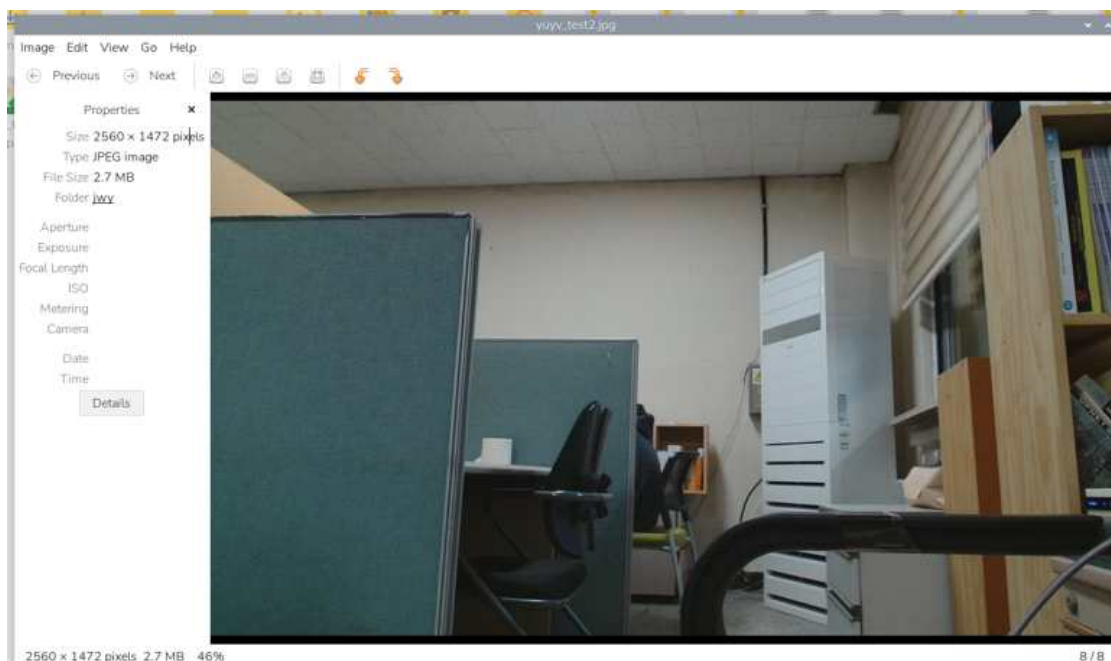
```
juw@raspberrypi:~$ fswebcam -d /dev/video0 -p MJPEG -r 1920x1080 --skip 20 --no-banner mjpg_test2.jpg
--- Opening /dev/video0...
Trying source module v4l2...
/dev/video0 opened.
No input was specified, using the first.
--- Capturing frame...
Skipping 20 frames...
Capturing 1 frames...
Captured 21 frames in 0.67 seconds. (31 fps)
--- Processing captured image...
Disabling banner.
Writing JPEG image to 'mjpg_test2.jpg'.
```

*MJPEG와 MJPG는 서로 같은 것을 가리킨다. fswebcam은 MJPEG만 지원(알아듣기)하기 때문에 MJPEG로 사용.



```
fswebcam -d /dev/video0 -p YUYV -r 2560x1472 -skip 20 -no-banner yuyv_test2.jpg
```

```
jwy@raspberrypi:~$ fswebcam -d /dev/video0 -p YUYV -r 2560x1472 --skip 20 --no-banner yuyv_test2.jpg
--- Opening /dev/video0...
Trying source module v4l2...
/dev/video0 opened.
No input was specified, using the first.
--- Capturing frame...
Skipping 20 frames...
Capturing 1 frames...
Captured 21 frames in 10.00 seconds. (2 fps)
--- Processing captured image...
Disabling banner.
Writing JPEG image to 'yuyv_test2.jpg'.
```



참고: Fps (frame per second):초당 프레임-끊김 없이 영상 볼 수 있음.

10~15fps면 똑똑 끊기는 느낌이 들고, 빠르게 지나가는 물체는 잔상 처럼 보입니다.

30fps면 일반적인 영상 통화나 TV 방송 수준으로, 일상적인 움직임을 보기에 충분합니다.

60 FPS 이상: 매우 부드럽게 느껴지며, 게임이나 정밀한 모니터링에서 필수적입니다.

--skip20: 초반 프레임을 20장 버리고 그 다음 프레임으로 촬영.
-> 웹캠을 막 열었을 때 자동 초점 등 다양한 사유로 인해 안정되지 않아서 첫 프레임이 흐리거나 어둡게 나올 수 있다. 그래서 안정된 뒤에 촬영하게 된다.

--no banner : 날짜나 시간, 해상도, 장치 정보 등 이러한 배너 글씨를 없애는 옵션임. -> 사진이 깔끔하게 저장된다.

yuyv는 2560x1472로 높은 해상도를 제공하였으나, 캡처 속도가 매우 느려 2 fps로 나옴. (10초 걸림) -실시간 처리에는 적합하지 않다.
mjpg는 1920x1080로 해상도가 yuyv보다 낮지만, 31fps로 매우 빠름.

yuyv는 정지사진 1장 화질을 최대한 좋게 뽑고 싶을 때 사용하는게 좋음. 프레임 속도 상관없을 때 사용하기 좋다.

mjpg는 실시간 처리에 유리하고 파일 크기도 작아서 라즈베리파이 -> 웹스트리밍->pc추론 하는 구조에선 mjpg가 더 적합함.

mjpg는 웹 스트리밍 및 향후 pc서버 기반 yolo 추론 입력 포맷으로 적합

참고) 화질은 해상도를 포함한 전체 품질로 노이즈나 조명, 압축 방식 등이 화질의 결정 요소이다.

즉 해상도는 화질을 결정하는 요소 중 하나. 해상도<화질

출처: [웹캠 사용법](#)

cheese(라이브 스트리밍)

```

alter_bin/ClusterGetVideoSink:clustergetvideosink10:
There may be a timestamping problem, or this computer is too slow.

(org.gnome.Cheese:3405): cheese-WARNING **: 2020-08-20 14:01: A lot of buffers are being dropped.: ../libs/gst/base/gstbasesink.c(3346): gst_base_sink_is_too_slow ():
alter_bin/ClusterGetVideoSink:clustergetvideosink10:
There may be a timestamping problem, or this computer is too slow.

(org.gnome.Cheese:3405): cheese-WARNING **: 2020-08-20 14:01: A lot of buffers are being dropped.: ../libs/gst/base/gstbasesink.c(3346): gst_base_sink_is_too_slow ():
alter_bin/ClusterGetVideoSink:clustergetvideosink12:
There may be a timestamping problem, or this computer is too slow.

(org.gnome.Cheese:3405): cheese-WARNING **: 2020-08-20 14:01: A lot of buffers are being dropped.: ../libs/gst/base/gstbasesink.c(3346): gst_base_sink_is_too_slow ():
alter_bin/ClusterGetVideoSink:clustergetvideosink14:
There may be a timestamping problem, or this computer is too slow.

(org.gnome.Cheese:3405): cheese-WARNING **: 2020-08-20 14:01: A lot of buffers are being dropped.: ../libs/gst/base/gstbasesink.c(3346): gst_base_sink_is_too_slow ():
alter_bin/ClusterGetVideoSink:clustergetvideosink15:
There may be a timestamping problem, or this computer is too slow.

(org.gnome.Cheese:3405): cheese-WARNING **: 2020-08-20 14:01: A lot of buffers are being dropped.: ../libs/gst/base/gstbasesink.c(3346): gst_base_sink_is_too_slow ():
alter_bin/ClusterGetVideoSink:clustergetvideosink18:
There may be a timestamping problem, or this computer is too slow.

(org.gnome.Cheese:3405): cheese-WARNING **: 2020-08-20 14:01: A lot of buffers are being dropped.: ../libs/gst/base/gstbasesink.c(3346): gst_base_sink_is_too_slow ():
gstvideosink0:
There may be a timestamping problem, or this computer is too slow.

(org.gnome.Cheese:3405): Cluster-CRITICAL **: 2020-08-20 14:01: Unable to create dummy unstream: No foreign surface, and wl_shm unsupported by the compositor

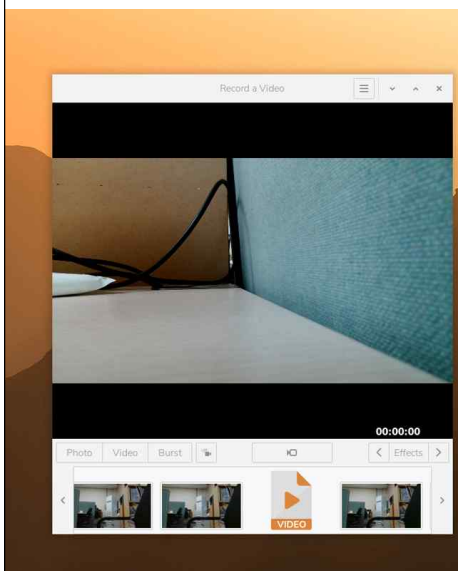
```

command에 cheese 라고 치면 바로 실행.

GUI 환경에서 실시간 스트리밍(라즈베리파이에서 실행)

cheese를 실행하면 USB 카메라의 실시간 화면

위 오류는 프레임 드롭 및 화면 출력하는데 경고 발생이어서 카메라 장치는 정상적으로 동작하였음. -> cheese 프로그램이 문제 원인(그래픽 출력 환경 호환성 문제가 있음)



출처: [라즈베리파이 카메라 연결 확인](#)

5. OpenCV,requests 설치(5 of 9)

라즈베리파이에서 USB 카메라 모듈을 Python이나 다른 언어로 활용하려면 OpenCV를 사용하는 것이 일반적이다.



opencv 정의랑 사용 이유 장단점 서술

opencv(open source computer vision library):이미지 및 영상 처리, 객체 인식, 카메라 제어 등 다양한 컴퓨터 비전 기능을 제공하는 오픈소스 라이브러리. python이나 java, c++ 등 다양한 언어를 지원하며, 실시간 영상 처리 및 영상 기반 응용 프로그램 개발에 활용된다.

장점: 실시간 영상 처리에 적합한 다양한 함수와 안정적인 카메라 제어 기능을 제공한다. python 환경에서 비교적 간단한 코드로 이미지 캡처, 해상도 조절, 프레임 처리, 영상 전처리 등을 수행할 수 있어서 개발 효율성이 높다.

단점: 기본 설정 상태에서 카메라 해상도나 포맷이 최적화되지 않을 경우 낮은 해상도로 영상이 저장될 수 있으며, 라즈베리파이와 같은 임베디드 환경에서 고해상도 실시간 처리할 시 성능 저하가 발생할 수 있어서 프로젝트 환경에 맞게 해상도 및 영상 포맷 설정이 필요함.

-> 웹캠으로부터 촬영된 이미지 저장 후 , yolo기반 객체 탐지 및 얼굴 인식 모델과 연계하기 위해 opencv를 사용하였다.

폴더 생성 및 가상환경 활성화

```
File Edit Tools Help
jwy@raspberrypi-5:~$ mkdir opencv_test
mkdir: cannot create directory 'opencv_test': File exists
jwy@raspberrypi-5:~$ cd opencv_test
jwy@raspberrypi-5:~/opencv_test$ python3 -m venv testenv
jwy@raspberrypi-5:~/opencv_test$ source testenv/bin/activate
(testenv) jwy@raspberrypi-5:~/opencv_test$ python -m pip install --upgrade pip
```

1. 프로젝트 폴더(opencv_test) 생성
2. 폴더로 이동
3. testenv 가상환경을 만들어서 가상환경 활성화

이후 opencv 관련 설치

`sudo apt install -y libopenblas-dev` os에 수학연산엔진 설치
(opencv 실행에 필요한 수학 연산 엔진)

```
(testenv) jwy@raspberrypi-5:~/opencv_test$ sudo apt install -y libopenblas-dev
libopenblas-dev is already the newest version (0.3.29+ds-3).
The following packages were automatically installed and are no longer required:
  alacarte gir1.2-gmenu-3.0 gnome-menus gtk-nop libgnome-menu-3.0 python3-gi-cairo retry
Use 'sudo apt autoremove' to remove them.

Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 43
```

`python -m pip install --upgrade pip`
`python -m pip install opencv-python`

```
(testenv) jwy@raspberrypi-5:~/opencv_test$ python -m pip install --upgrade pip
Looking in indexes: https://pypi.org/simple, https://www.piwheels.org/simple
Requirement already satisfied: pip in ./testenv/lib/python3.13/site-packages (26.0.1)
(testenv) jwy@raspberrypi-5:~/opencv_test$ python -m pip install opencv-python
Looking in indexes: https://pypi.org/simple, https://www.piwheels.org/simple
Requirement already satisfied: opencv-python in ./testenv/lib/python3.13/site-packages (4.13.0.92)
Requirement already satisfied: numpy>=2 in ./testenv/lib/python3.13/site-packages (from opencv-python) (2.4.4)
```

opencv 관련해서 pip버전을 업그레이드 하고, opencv 설치
opencv는 기본적으로 python3를 기반으로 사용한다.

python -c "import cv2; > print(cv2.__version__)"

```
(testvenv) jwy@raspberrypi-5:~/opencv_test $ python -c "import cv2;
> print(cv2.__version__)"
4.13.0
```

python 환경에서 cv2 모듈 import 성공 후 opencv 버전이 4.13.0
인 것을 확인

nano live_test.py - 코드 생성

python live_test.py - 코드 실행

```
File Edit Tabs Help
GNU nano 8.4 live_test.py
import cv2

cap = cv2.VideoCapture(0)

if not cap.isOpened():
    print("unable to open camera.")
    exit()

while True:
    ret, frame = cap.read()
    if not ret:
        print("unable to retrieve the frame")
        break

    cv2.imshow('USB Camera', frame)

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

전체 코드

실행 중인 화면 기록

6. OpenCV 기반 이미지 캡처 및 전송 코드 작성 (6 of 9)

라즈베리파이 이미지 캡처

[nano capture_test.py](#)

[python capture_test.py](#)

```
GNU nano 8.4 capture_test.py
import cv2

cap = cv2.VideoCapture(0)

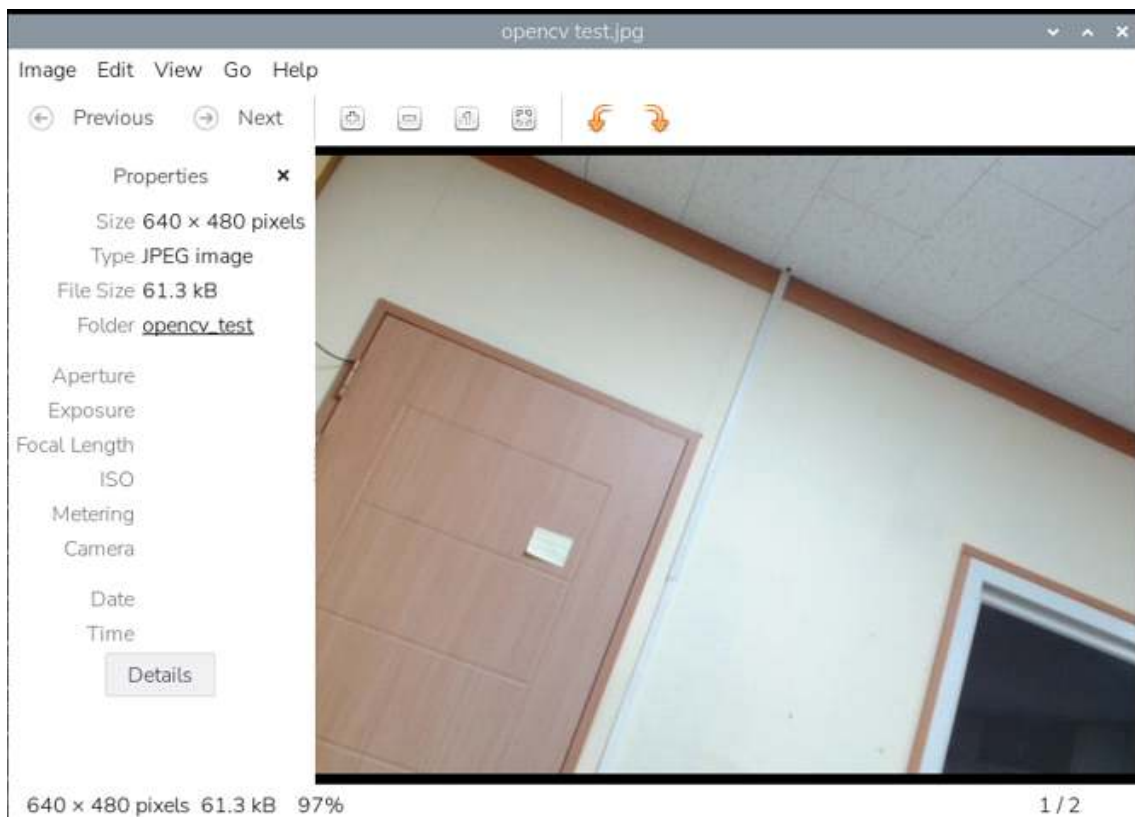
if not cap.isOpened():
    print("unable to open camera.")
    exit()

ret, frame = cap.read()

if ret:
    cv2.imwrite("opencv test.jpg", frame)
    print("saved: opencv test.jpg")
else:
    print("capture failed")

cap.release()
```

전체 코드



opencv_test라는 이미지가 캡처됨.

해상도가(640x480) 기본 해상도로 캡처 됨.

이미지 파일 PC서버에 전송

nano send_image.py

python send_image.py



```
File Edit Tabs Help
GNU nano 8.4 send_image.py
import requests

url = "http://192.168.1.6:8000/upload"

with open("opencv test.jpg", "rb") as f:
    files = {"file": ("opencv test.jpg", f, "image/jpeg")}

    response = requests.post(url, files=files)

print(response.json())
```

7. PC 서버 기록과 저장 폴더를 확인하여 이미지 전송 성공 여부 검증 (7 of 9)

사진 저장과 서버 전송 두 코드를 이제 하나의 코드로 합침.

nano capture_and_send.py

python capture_and_send.py

```
GNU nano: 8.4 capture_and_send.py
import cv2
import requests

URL = "http://192.168.1.6:8080/upload"

cap = cv2.VideoCapture(0)

cap.set(cv2.CAP_PROP_FOURCC, cv2.VideoWriter_fourcc('M','J','P','G'))
cap.set(cv2.CAP_PROP_FRAME_WIDTH, 1920)
cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 1080)

if not cap.isOpened():
    print("unable to open camera")
    exit()

ret, frame = cap.read()

if ret:
    cv2.imwrite("send_test.jpg", frame)
    print("capture success")

    with open("send_test.jpg", "rb") as f:
        files = {"file": ("from_pi.jpg", f, "image/jpeg")}

        try:
            response = requests.post(URL, files=files)
            print("server response:", response.json())

        except Exception as e:
            print("transmission failed", e)

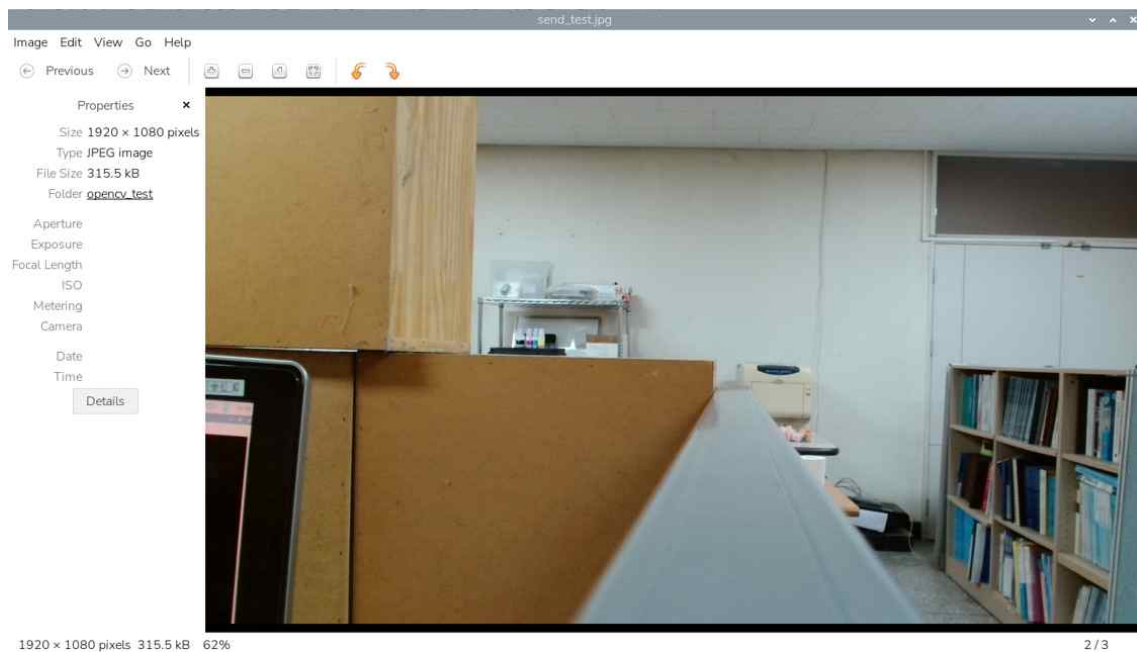
    else:
        print("frame acquisition failed")

cap.release()
```

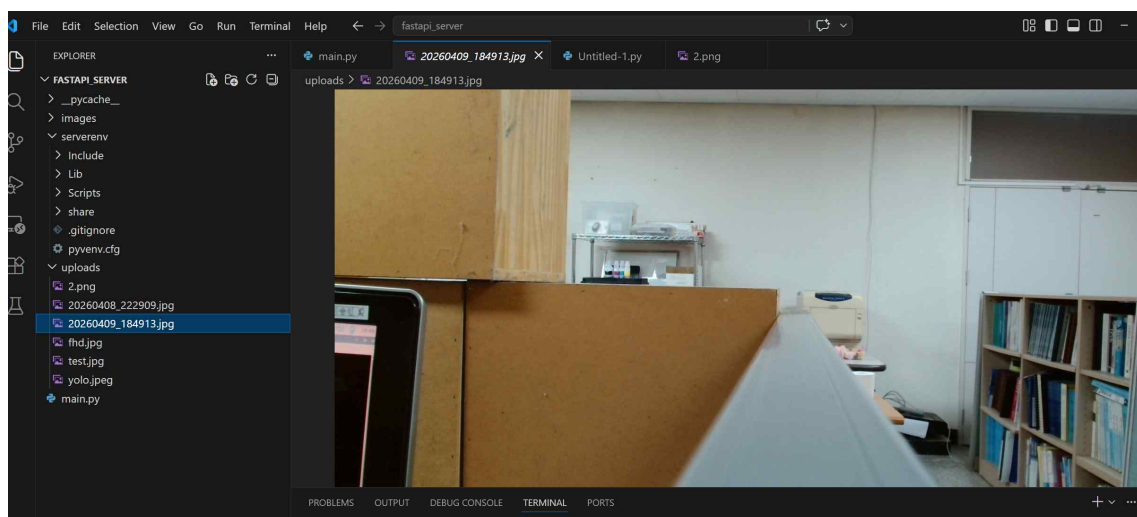
기본 해상도에서 mjpg 포맷과 프레임 크기를 1920x1080으로 카메라 설정을 추가한다.(cap.set 세 줄)- 화질 개선
이하는 동일.

```
(testvenv) jwy@raspberrypi-5:~/opencv_test $ nano capture_and_send.py
(testvenv) jwy@raspberrypi-5:~/opencv_test $ python capture_and_send.py
capture success
server response: {'status': 'success', 'filename': '20260409_184913.jpg'}
```

캡처가 성공적으로 확인되었고 서버에 정상적으로 보내짐.



이미지가 서버로 정상적으로 전송됨
(pc서버 사진)



8. 정적 이미지 전송 이후 실시간 영상 확인을 위한 MJPG 스트리밍 환경 검증(8 of 9)

앞 단계에서는 OpenCV를 이용하여 단일 이미지를 캡처-> 해당 이미지를 PC FastAPI 서버로 전송하는 기능을 구현하였다.

이를 통해 라즈베리파이에서 수집한 영상 데이터가 PC 서버로 정상 전달되는 것을 확인할 수 있었다.

최종 시스템은 단일 이미지가 아니라 시간에 따라 연속적으로 입력되는 영상 프레임을 기반으로 사람 감지, 얼굴 인식 및 미등록자 판별 등을 수행해야 한다.(한 장씩 전송하는 방식은 특정 시점의 장면 확인에는 적합하지만, 사람의 이동이나 출입 상황을 연속적으로 확인해야 하는 실시간 감시에는 한계가 있다.)

따라서 정적 이미지 전송 기능을 확인한 후, 다음 단계로 라즈베리파이에서 웹캠 영상을 연속 프레임 형태로 제공할 수 있는지 검증하기 위해 라이브 스트리밍 환경을 추가로 구성하였다.

mjpg 스트리밍

MJPG는 여러 장의 JPEG 프레임을 연속적으로 전송하여 브라우저에서 영상처럼 보이게 하는 방식이다. 구현이 단순하고 웹 브라우저에서 바로 확인하기 쉽지만, 압축 효율이 낮아 대역폭을 비교적 많이 사용할 수 있다

빠르게 웹에서 라이브 화면 볼 수 있음.

전체 구조도



구조도에서 input_uvc.so는 라즈베리파이에 연결된 USB 웹캠 (/dev/video0)으로부터 영상 프레임을 읽어오는 입력 플러그인이다. mjpg-streamer는 입력된 프레임을 MJPEG 방식으로 연속 처리하고, output_http.so는 처리된 프레임을 HTTP 방식으로 외부 브라우저에 제공한다. 따라서 사용자는 같은 네트워크에 연결된 PC 또는 모바일 브라우저에서 `http://192.168.1.8:8080`으로 접속하여 라즈베리파이 웹캠의 실시간 영상을 확인할 수 있다.

1) 작업폴더 만들기

```
(testvenv) jwy@raspberrypi-5:~/opencv_test $ mkdir -p ~/mjpg_streamer_test
(testvenv) jwy@raspberrypi-5:~/opencv_test $ cd ~/mjpg_streamer_test
```

2) 필요한 패키지 설치하기

`sudo apt install -y git build-essential libjpeg62-turbo-dev`

```
(testvenv) jwy@raspberrypi-5:~/mjpg_streamer_test $ sudo apt install -y git cmake build-essential libjpeg62-turbo-dev
git is already the newest version (1:2.47.3-0+deb13u1).
cmake is already the newest version (3.31.6-2).
build-essential is already the newest version (12.12).
The following packages were automatically installed and are no longer required:
  alacarte gir1.2-gmenu-3.0 gnome-menus gtk-nop libgnome-menu-3-0 python3-gi-cairo retry
Use 'sudo apt autoremove' to remove them.

Installing:
  libjpeg62-turbo-dev

Summary:
  Upgrading: 0, Installing: 1, Removing: 0, Not Upgrading: 44
  Download size: 293 kB
  Space needed: 800 kB / 110 GB available

Get:1 http://deb.debian.org/debian trixie/main arm64 libjpeg62-turbo-dev arm64 1:2.1.5-4 [293 kB]
Fetched 293 kB in 0s (1,119 kB/s)
Selecting previously unselected package libjpeg62-turbo-dev:arm64.
(Reading database ... 147386 files and directories currently installed.)
Preparing to unpack .../libjpeg62-turbo-dev_1%3a2.1.5-4_arm64.deb ...
Unpacking libjpeg62-turbo-dev:arm64 (1:2.1.5-4) ...
Setting up libjpeg62-turbo-dev:arm64 (1:2.1.5-4) ...
```

git = 소스코드 다운로드

cmake= 빌드 준비

build-essential= 컴파일 도구

libjpeg62-turbo-dev=jpeg 처리 관련 라이브러리

Github에서 mjpg-streamer 내려받고 make로 직접 빌드하려면 다음 명령어를 입력하면 됨.

3) 소스 내려받기

`git clone https://github.com/jacksonliam/mjpg-streamer.git`

```
(testvenv) jwy@raspberrypi-5:~ $ git clone ^C
(testvenv) jwy@raspberrypi-5:~ $ git clone https://github.com/jacksonliam/mjpg-streamer.git
Cloning into 'mjpg-streamer'...
remote: Enumerating objects: 2964, done.
remote: Counting objects: 100% (861/861), done.
remote: Compressing objects: 100% (193/193), done.
remote: Total 2964 (delta 696), reused 668 (delta 668), pack-reused 2103 (from 1)
Receiving objects: 100% (2964/2964), 3.42 MiB | 8.99 MiB/s, done.
Resolving deltas: 100% (1913/1913), done.
```

github에서 mjpg-streamer 소스코드를 라즈베리파이로 가져오는 단계

1.cd mjpg-streamer-experimental

2.make

```
(testvenv) jwy@raspberrypi-5:~/mjpg-streamer $ cd mjpg-streamer-experimental
(testvenv) jwy@raspberrypi-5:~/mjpg-streamer/mjpg-streamer-experimental $ make
[ -d _build ] || mkdir _build
[ -f _build/Makefile ] || (cd _build && cmake -DCMAKE_BUILD_TYPE=Release ..)
CMake Deprecation Warning at CMakeLists.txt:1 (cmake_minimum_required):
  Compatibility with CMake < 3.10 will be removed from a future version of
  CMake.

  Update the VERSION argument <min> value. Or, use the <min>...<max> syntax
  to tell CMake that the project requires at least <min> but has been updated
  to work with policies introduced by <max> or earlier.

-- The C compiler identification is GNU 14.2.0
-- Detecting C compiler ABI info
-- Detecting C compiler ABI info - done
-- Check for working C compiler: /usr/bin/cc - skipped
-- Detecting C compile features
-- Detecting C compile features - done
CMAKE_BUILD_TYPE = Release
-- Looking for include file sys/inotify.h
-- Looking for include file sys/inotify.h - found
CMake Warning at plugins/input_opencv/CMakeLists.txt:4 (find_package):
  By not providing "FindOpenCV.cmake" in CMAKE_MODULE_PATH this project has
  asked CMake to find a package configuration file provided by "OpenCV", but
  CMake did not find one.

  Could not find a package configuration file provided by "OpenCV" with any
  of the following names:

    OpenCVConfig.cmake
    opencv-config.cmake

  Add the installation prefix of "OpenCV" to CMAKE_PREFIX_PATH or set
  "OpenCV_DIR" to a directory containing one of the above files.  If "OpenCV"
```

1.실제 컴파일이 이루어지는 폴더로 이동

2.이후 소스코드를 실행가능한 프로그램과 플러그인 파일로 빌드(=컴파일)

참조) 플러그인 파일은 본체 프로그램(mjpg-streamer)에 붙여서 기능을 추가하는 부품 파일

- 본체 혼자서는 카메라를 어떻게 읽을지 어디로 내보낼지 모르니 플러그인 파일을 붙여서 기능을 정함.

sudo make install

```

[ 91%] Linking C shared library output_rtp.so
[ 91%] Built target output_rtp
[ 95%] Building C object plugins/output_udp/CMakeFiles/output_udp.dir/output_udp.c.o
[100%] Linking C shared library output_udp.so
[100%] Built target output_udp
make[1]: Leaving directory '/home/jwy/mjpg-streamer/mjpg-streamer-experimental/_build'
(testvenv) jwy@raspberrypi-5:~/mjpg-streamer/mjpg-streamer-experimental $ sudo make install
make -C _build install
make[1]: Entering directory '/home/jwy/mjpg-streamer/mjpg-streamer-experimental/_build'
[ 13%] Built target mjpg_streamer
[ 21%] Built target input_file
[ 39%] Built target input_http
[ 69%] Built target input_uvc
[ 69%] Built target output_file
[ 82%] Built target output_http
[ 91%] Built target output_rtp
[100%] Built target output_udp
Install the project.
-- Install configuration: "Release"
-- Installing: /usr/local/bin/mjpg_streamer
-- Set non-toolchain portion of runtime path of "/usr/local/bin/mjpg_streamer" to "/usr/local/lib/mjpg-streamer"
-- Installing: /usr/local/share/mjpg-streamer/www
-- Installing: /usr/local/share/mjpg-streamer/www/index.html
-- Installing: /usr/local/share/mjpg-streamer/www/favicon.png
-- Installing: /usr/local/share/mjpg-streamer/www/jquery.js
-- Installing: /usr/local/share/mjpg-streamer/www/flix.css
-- Installing: /usr/local/share/mjpg-streamer/www/javascript_simple.html
-- Installing: /usr/local/share/mjpg-streamer/www/sidebarbg.gif
-- Installing: /usr/local/share/mjpg-streamer/www/jquery.ui.custom.css
-- Installing: /usr/local/share/mjpg-streamer/www/style.css
-- Installing: /usr/local/share/mjpg-streamer/www/static_simple.html
-- Installing: /usr/local/share/mjpg-streamer/www/static.html
-- Installing: /usr/local/share/mjpg-streamer/www/videoLAN.html
-- Installing: /usr/local/share/mjpg-streamer/www/example.jpg
-- Installing: /usr/local/share/mjpg-streamer/www/LICENSE.txt
-- Installing: /usr/local/share/mjpg-streamer/www/jquery.ui.core.min.js
-- Installing: /usr/local/share/mjpg-streamer/www/java_simple.html
-- Installing: /usr/local/share/mjpg-streamer/www/jquerySpinBtn.css
-- Installing: /usr/local/share/mjpg-streamer/www/javascript_motionDetection.html
-- Installing: /usr/local/share/mjpg-streamer/www/java.html
-- Installing: /usr/local/share/mjpg-streamer/www/javascript.html
-- Installing: /usr/local/share/mjpg-streamer/www/jquery.ui.tabs.min.js
-- Installing: /usr/local/share/mjpg-streamer/www/control.htm
-- Installing: /usr/local/share/mjpg-streamer/www/bodybg.gif
-- Installing: /usr/local/share/mjpg-streamer/www/java_control.html
-- Installing: /usr/local/share/mjpg-streamer/www/spinbtn_updn.gif
-- Installing: /usr/local/share/mjpg-streamer/www/functions.js
-- Installing: /usr/local/share/mjpg-streamer/www/jquerySpinBtn.js
-- Installing: /usr/local/share/mjpg-streamer/www/stream_simple.html
-- Installing: /usr/local/share/mjpg-streamer/www/jquery.ui.widget.min.js
-- Installing: /usr/local/share/mjpg-streamer/www/favicon.ico
-- Installing: /usr/local/share/mjpg-streamer/www/stream.html
-- Installing: /usr/local/share/mjpg-streamer/www/CamOjoia.jar
-- Installing: /usr/local/share/mjpg-streamer/www/rotateIcons.png
-- Installing: /usr/local/share/mjpg-streamer/www/jquery.rotate.js
-- Installing: /usr/local/lib/mjpg-streamer/input_file.so
-- Installing: /usr/local/lib/mjpg-streamer/input_http.so
-- Installing: /usr/local/lib/mjpg-streamer/input_uvc.so
-- Installing: /usr/local/lib/mjpg-streamer/output_file.so
-- Installing: /usr/local/lib/mjpg-streamer/output_http.so
-- Installing: /usr/local/lib/mjpg-streamer/output_rtp.so
-- Installing: /usr/local/lib/mjpg-streamer/output_udp.so
make[1]: Leaving directory '/home/jwy/mjpg-streamer/mjpg-streamer-experimental/_build'
(testvenv) jwy@raspberrypi-5:~/mjpg-streamer/mjpg-streamer-experimental $

```

시스템에서 사용할 수 있도록 설치

4) 실행

1. export LD_LIBRARY_PATH=.

2. ./mjpg_streamer -i "input_uvc.so -d /dev/video0 -r 1920x1080 -f 15" -o "output_http.so -w ./www -p 8080"

```

(testvenv) jw@raspberrypi-5:~/mjpg-streamer/mjpg-streamer-experimental $ export LD_LIBRARY_PATH=.
(testvenv) jw@raspberrypi-5:~/mjpg-streamer/mjpg-streamer-experimental $ ./mjpg_streamer -i "input_uvc.so -d /dev/video0 -r 1920x1080 -f 15" -o "output_http.so -w ./www -p 8080"
MJPEG Streamer Version: git rev: 310b29f4a94c46b52b26c4b7b6e5cf24e532af39
i: Using V4L2 device.: /dev/video0
i: Desired Resolution: 1920 x 1080
i: Frames Per Second.: 15
i: Format.....: JPEG
i: TV-Norm.....: DEFAULT
UVCIOC_CTRL_ADD - Error at Pan (relative): Inappropriate ioctl for device (25)
UVCIOC_CTRL_ADD - Error at Tilt (relative): Inappropriate ioctl for device (25)
UVCIOC_CTRL_ADD - Error at Pan Reset: Inappropriate ioctl for device (25)
UVCIOC_CTRL_ADD - Error at Tilt Reset: Inappropriate ioctl for device (25)
UVCIOC_CTRL_ADD - Error at Pan/tilt Reset: Inappropriate ioctl for device (25)
UVCIOC_CTRL_ADD - Error at Focus (absolute): Inappropriate ioctl for device (25)
UVCIOC_CTRL_MAP - Error at Pan (relative): Inappropriate ioctl for device (25)
UVCIOC_CTRL_MAP - Error at Tilt (relative): Inappropriate ioctl for device (25)
UVCIOC_CTRL_MAP - Error at Pan Reset: Inappropriate ioctl for device (25)
UVCIOC_CTRL_MAP - Error at Tilt Reset: Inappropriate ioctl for device (25)
UVCIOC_CTRL_MAP - Error at Pan/tilt Reset: Inappropriate ioctl for device (25)
UVCIOC_CTRL_MAP - Error at Focus (absolute): Inappropriate ioctl for device (25)
UVCIOC_CTRL_MAP - Error at LED1 Mode: Inappropriate ioctl for device (25)
UVCIOC_CTRL_MAP - Error at LED1 Frequency: Inappropriate ioctl for device (25)
UVCIOC_CTRL_MAP - Error at Disable video processing: Inappropriate ioctl for device (25)
UVCIOC_CTRL_MAP - Error at Raw bits per pixel: Inappropriate ioctl for device (25)
o: www-folder-path.....: ./www/
o: HTTP TCP port.....: 8080
o: HTTP Listen Address...: (null)
o: username:password....: disabled
o: commands.....: enabled
$
^CSetting signal to stop
i: cleaning up resources allocated by input thread
force cancellation of threads and cleanup resources
o: cleaning up resources allocated by server thread #00
done

```

1. 라이브러리 경로 설정

2. mjpg-streamer 실행

-input_uvc.so :usb 웹캠 입력 플러그인 파일(카메라에서 영상 읽어옴)

-output_http.so:http 웹 출력 플러그인 파일(영상을 웹으로 보내줌)

-w ./www:기본 웹 페이지 폴더 지정

-p 8080:포트 번호 설정

1920x1080으로 했을 때 프레임 끊김이 자주 발생 프레임을 10(-f 15)으로 하여도 끊김 자주발생

참고) .so는 리눅스에서 공유 라이브러리 파일로 실행 파일 본체가 있고 필요할 때 불러다 쓰는 기능이 묶여있는 파일이다.

이후 1280x720으로 조정

`./mjpg_streamer -i "input_uvc.so -d /dev/video0 -r 1280x720 -f 30" -o "output_http.so -w ./www -p 8080"`

```
(testvenv) jwy@raspberrypi-5:~/mjpg-streamer/mjpg-streamer-experimental$ ./mjpg_streamer -i "input_ufv.so -d /dev/video0 -r 1280x720 -f 30" -o "output_http.so -w ./www -p 8080"
MJPEG Streamer Version: git rev: 310b29f4a94c46852b26c4b7b6e5cf24e532af39
1: Using V4L2 device.: /dev/video0
1: Desired Resolution: 1280 x 720
1: Frames Per Second.: 30
1: Format.....: JPEG
1: TV-Norm.....: DEFAULT
UVCIOC_CTRL_ADD - Error at Pan (relative): Inappropriate ioctl for device (25)
UVCIOC_CTRL_ADD - Error at Tilt (relative): Inappropriate ioctl for device (25)
UVCIOC_CTRL_ADD - Error at Pan Reset: Inappropriate ioctl for device (25)
UVCIOC_CTRL_ADD - Error at Tilt Reset: Inappropriate ioctl for device (25)
UVCIOC_CTRL_ADD - Error at Pan/Tilt Reset: Inappropriate ioctl for device (25)
UVCIOC_CTRL_ADD - Error at Focus (absolute): Inappropriate ioctl for device (25)
UVCIOC_CTRL_MAP - Error at Pan (relative): Inappropriate ioctl for device (25)
UVCIOC_CTRL_MAP - Error at Tilt (relative): Inappropriate ioctl for device (25)
UVCIOC_CTRL_MAP - Error at Pan Reset: Inappropriate ioctl for device (25)
UVCIOC_CTRL_MAP - Error at Tilt Reset: Inappropriate ioctl for device (25)
UVCIOC_CTRL_MAP - Error at Pan/Tilt Reset: Inappropriate ioctl for device (25)
UVCIOC_CTRL_MAP - Error at Focus (absolute): Inappropriate ioctl for device (25)
UVCIOC_CTRL_MAP - Error at LED1 Mode: Inappropriate ioctl for device (25)
UVCIOC_CTRL_MAP - Error at LED1 Frequency: Inappropriate ioctl for device (25)
UVCIOC_CTRL_MAP - Error at Disable video processing: Inappropriate ioctl for device (25)
UVCIOC_CTRL_MAP - Error at Raw bits per pixel: Inappropriate ioctl for device (25)
o: www-folder-path.....: ./www/
o: HTTP TCP port.....: 8080
o: HTTP Listen Address...: (null)
o: username:password....: disabled
o: commands.....: enabled
^Csetting signal to stop
1: cleaning up resources allocated by input thread
force cancellation of threads and cleanup resources
o: cleaning up resources allocated by server thread #00
done
```

프레임을 30으로 하였는데, 프레임 드랍이 거의 없음.

라즈베리파이 ip주소:192.168.1.8

<http://192.168.1.8:8080> 으로 브라우저 접속

(실행 및 설명 영상)

mjpg-streamer는 시연용. -yolo결과 붙이기엔 어려움.

최종 YOLO 결과 표시에는 한계가 있어, 이후에는 Python 웹 서버 스트리밍으로 확장할 계획입니다.

9. FastAPI와 MySQL 연결(9 of 9)

MySQL: 데이터 저장 및 관리에 사용되는 오픈 소스 관계형 데이터베이스 관리 시스템(RDBMS)이다. 안정성, 성능, 확장성, 사용 편의성을 갖춘 MySQL은 개발자들에게 널리 사용되고 있다.

관계형데이터베이스: 데이터가 적절히 분리되고 연결된 데이터베이스

SQL과의 차이점: SQL은 데이터베이스에 저장된 데이터를 조작하는데 사용되는 프로그래밍 언어의 일종인 구조화된 쿼리 언어의 약자이다. MySQL은 SQL 언어를 사용하여 데이터베이스의 데이터를 관리 및 쿼리하므로 이름에 해당 약자를 사용한다. MySQL RDBMS에 데이터가 저장되어 있는 경우 간단한 SQL 프롬프트를 작성해 데이터를 추가, 검색, 분석할 수 있다.

-> MySQL은 사용이 간단하고, Python 등 다양한 기술 플랫폼 및 프로그래밍 언어와 폭넓은 호환성을 제공한다.

SQLAlchemy: 파이썬 SQL 툴킷이면서 파이썬 프로그래밍 언어를 위한 ORM이라고 한다. ORM(Object Relational Mapping)은 객체-관계 매핑이라는 의미인데 즉 객체와 관계형 데이터베이스를 연결해주는 것이다.

<연결과정(main.py, db.py, model.py로 구분해서 연결)>

구분한 이유: main.py에 다 넣어도 되지만 코드가 길어져서 이후에 헛갈릴 가능성이 있어 나눴다.

db.py: MySQL 데이터베이스에 접속하고 작업할 수 있도록 연결 환경을 설정하는 파일

model.py: DB에 저장할 데이터의 구조와 형식을 정의하는 파일

main.py-> 저장된 데이터를 조회, 생성, 수정, 삭제하는 FastAPI 서버 기능을 구현한 파일

(1)db.py

DB 연결 및 기능 사용을 위한 모듈 import	<pre>from sqlalchemy import create_engine</pre> ->DB 연결(엔진) 생성 <pre>from sqlalchemy.ext.declarative import declarative_base</pre> ->테이블 구조 정의를 위한 Base 클래스 생성 <pre>from sqlalchemy.orm import sessionmaker, scoped_session</pre> ->sessionmaker: insert, select, update, delete 수행 ->scoped_session: 다중 사용자 요청 환경에서 세션을 안전하게 관리
DB 접속 정보 설정	<pre>user_name = "OJH" user_pwd = "1234" db_host = "127.0.0.1" db_name = "capstone"</pre>
DB 연결 문자 열 생성	<pre>DATABASE = f"mysql+pymysql://{user_name}:{user_pwd}@{db_host}/{db_name}?charset=utf8"</pre> ----- ---- ->pymysql: python과 MySQL과 통신하게 해주는 도구 ->charset=utf8: 한글 깨짐 방지
DB 연결 통로 (데이터베이스 엔진)	<pre>ENGINE = create_engine(DATABASE, echo=True)</pre>

	----- 위에서 연결한 DATABASE 포함함 ENGINE: 실제 DB 연결을 관리함 echo=True: 실행되는 SQL을 화면에 보여줘서 디버깅하기 쉽게 해줌
DB 작업 수행 객체 (세션 생성)	<pre>session = scoped_session(sessionmaker(autocommit=False, autoflush=False, bind=ENGINE))</pre> ----- DB작업 실제로 수행하는 객체를 만드는 부분임 맨 처음 세 번째 코드의 scoped_session으로 작성한다. 세션으로 하는 일: 조회, 저장, 수정, 삭제
테이블 정의를 위한 Base 클 래스	<pre>Base = declarative_base() Base.query = session.query_property()</pre> ----- 뒤에서 만들 테이블 클래스의 부모클래스 만듦 Base 또한 맨 처음 불러온 declarative_base()를 사용한다.

(2) model.py

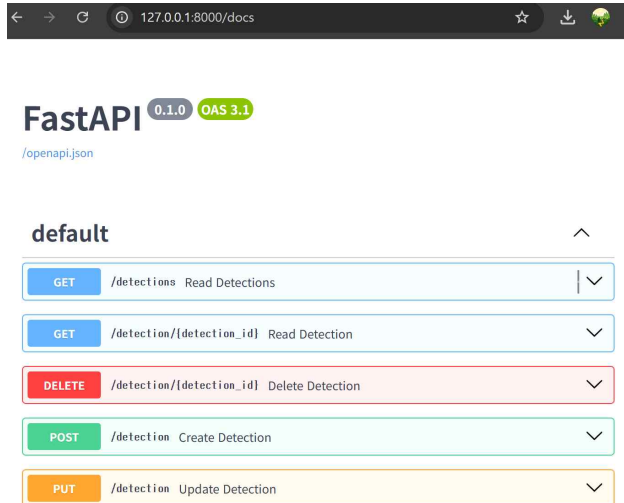
테이블 생성에 필요한 모듈 import	<pre> from sqlalchemy import Column, Integer, String from pydantic import BaseModel from db import Base </pre>
Detection 테이블 정의 (DB에 저장될 구조)	<pre> class DetectionTable(Base): __tablename__ = "detection" #DB테이블 이름 id = Column(Integer, primary_key=True, autoincrement=True) event_time = Column(String(50), nullable=False) camera_id = Column(Integer) image_path = Column(String(50), nullable=False) </pre> ----- id: 각 기록 구분 event_time: 언제 찍혔는지 camera_id: 카메라 번호(2개의 카메라 사용하기 때문) image_path: 이미지 파일 경로 위의 내용을 담는 클래스를 만들었다. id는 자동으로 증가되도록 autoincrement 값을 True로 설정, 각 정보들은 꼭 필요한 정보기에 nullable 값을 False로 설정
API 데이터 입출력 형식 정의	<pre> class Detection(BaseModel): id: int event_time: str camera_id: int image_path: str </pre>

(3) main.py

FastAPI 및 필요한 모듈 import	<pre> from fastapi import FastAPI from typing import List from db import session from model import DetectionTable, Detection -----FastAPI, 수정동 작을 위해서 List를 읽는 코드, 위의 session, DetectionTable, Detection 클래스를 불러온다. </pre>
FastAPI 애플리케이션 생성	<pre> app = FastAPI() </pre>
전체 데이터 조회 -> 모든 탐지 데이터 조회	<pre> @app.get("/detections") def read_detections(): detections = session.query(DetectionTable).all() return detections -----Det ectionTable 리스트 전체를 불러온다 </pre>
개별 데이터 조회 -> 특정 ID 데이터 조회	<pre> @app.get("/detection/{detection_id}") def read_detection(detection_id: int): detection = session.query(DetectionTable).filter(DetectionT able.id == detection_id).first() return detection -----각 탐 지 결과의 id에 맞춰 개별 결과를 불러온다. DetectionTable 클래스에 저장되어있는 id를 입력한 (찾고싶은) id와 비교한 후 가져오게된다. all이 아 닌 id를 비교하는 filter 조건으로 필요한 정보만을 가져올 수 있다. </pre>
데이터 생성 (Create)	<pre> @app.post("/detection") def create_detection(event_time: str, camera_id: int, image_path: str): detection = DetectionTable(event_time=event_time, </pre>

	<pre> camera_id=camera_id, image_path=image_path) session.add(detection) session.commit() return {"message": f"{event_time} created.."} -----dete ction의 데이터 생성 </pre>
데이터 수정 (Update) → 기존 데이터 수정	<pre> @app.put("/detection") def update_detection(detections: List[Detection]): for i in detections: detection = session.query(DetectionTable).filter(DetectionT able.id == i.id).first() if detection: detection.event_time = i.event_time detection.camera_id = i.camera_id detection.image_path = i.image_path session.commit() return {"message": "detections updated."} </pre>
데이터 삭제 (Delete) → 특정 데이터 삭제	<pre> @app.delete("/detection/{detection_id}") def delete_detection(detection_id: int): session.query(DetectionTable).filter(DetectionT able.id == detection_id).delete() session.commit() return {"message": f"detection {detection_id} deleted."} </pre>

에러 발생-> POST로 값을 입력했지만 저장이 되지 않음



오류원인 검색 결과->

본 시스템에서 발생한 500 Internal Server Error의 원인은 다음과 같이 분석된다.

1) 데이터베이스 테이블 미생성

MySQL 데이터베이스 내에 detection 테이블이 생성되지 않은 상태에서 데이터 삽입을 시도하여 오류가 발생했을 가능성이 있다.

2) ORM 모델과 DB 스키마 불일치

SQLAlchemy에서 정의한 DetectionTable 모델과 실제 MySQL 테이블 구조가

일치하지 않을 경우, 데이터 삽입 과정에서 예외가 발생할 수 있다.

3) 서버 코드 내부 오류 (오타 및 설정 문제)

코드 내 오타(nullable 오타 등) 또는 잘못된 컬럼 설정,

세션 처리 문제 등으로 인해 서버 내부에서 예외가 발생했을 가능성이 있다.

-> 향후 오류 분석 및 수정 과정을 통해 정상적인 데이터 저장 기능을 구현할 예정이다.

결과 및 결론: 라즈베리파이 5와 usb 웹캠을 이용하여 카메라 입력 환경을 구축하였고, opencv를 이용한 이미지 캡처 및 pc Fastapi 서버 업로드 기능을 구현하였습니다. 또한 mjpg-streamer를 이용한 웹 스트리밍 환경을 구성하여 브라우저에서 실시간 영상 확인이 가능함을 검증하였습니다.

-> 영상 입력, 이미지 전송, 웹 스트리밍까지의 파이프라인을 구축하는 것이 완료되었으며, 실시간 처리용 포맷으로는 mjpg가 가장 적합한 것으로 판단하였습니다. 이후에는 pc서버에서 yolo 객체 탐지와 얼굴 인식 기능을 결합하여 최종 침입 감지 시스템으로 확장 할 것입니다.

출처 : [mjpg-streamer github 코드:](#)

출처: [mjpg-streamer](#)